All tools Edit Export E-Sign

Elements of Cache Design

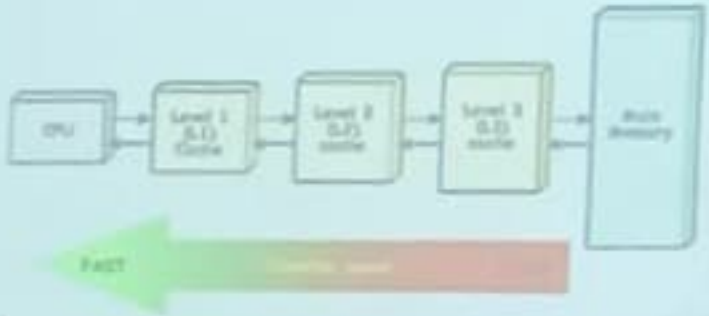
- Size
  - Cache Size,
  - Block Size
  - Line Size
- Addressing
  - Logical
  - Physical
- Mapping Function
  - Direct,
  - Associative,
  - Set Associative
- Write Policy
  - Write through,
  - Write back,
  - Write once
- Cache Levels/Types
  - Single or Multi-level
  - Unified or Split
- Replacement Algorithm
  - Least recently used
  - First in first out
  - Least frequently used
  - Random

Cache Hit and Miss

- When the CPU needs instruction/data, it immediately checks in cache memory whether it has data or not.
- If the reference is present it results in **cache hits**, else **cache miss.**
- Data is not in cache memory so it retrieves data from main memory.
- Cache misses can slow down performance, so computer systems use caching strategies to minimize their impact.

Cache Misses

- **Classifying Misses: 3 Cs**
  - Compulsory—Misses in even an Infinite Cache
  - Capacity—Misses in Fully Associative Size X Cache
  - Conflict—Misses in N-way Associative, Size X Cache
- *More recent, 4th "C":*
  - Coherence - Misses caused by cache coherence.



Compulsory Cache Miss

- *These misses occur when the first access to a cache block (line) happens (unless mechanism like prefetching is used)*
- It is also known as *cold start misses* or *first references misses*.
- Block must be brought into the cache.
- Compulsory misses occur same in all types of direct mapped, k-way set associat
  and fully associative caches.

Cache Prefetching

- *Cache line prefetching is a technique used in computer processors to improve memory access performance.*
- It involves fetching multiple contiguous cache lines from memory into the processor's cache in advance, anticipating that they will be needed in the near future.
- *Cache line prefetching takes advantage of the principle of spatial locality.*
- By fetching contiguous cache lines, which are typically 64 bytes or larger, the processor can reduce the latency of subsequent memory accesses.

Cache Prefetching

- Cache line prefetching is a technique used in computer processors to improve memory access performance.
- It involves fetching multiple contiguous cache lines from memory into the processor's cache in advance, anticipating that they will be needed in the near future.
- Cache line prefetching takes advantage of the principle of spatial locality.
- By fetching contiguous cache lines, which are typically 64 bytes or larger, the processor can reduce the latency of subsequent memory accesses.

All tools Edit Export E-Sign

Cache Prefetching: Classifications

- **Hardware prefetching**
- **Software prefetching**
- **Compiler-assisted prefetching**

- Cache line prefetching can improve performance by reducing memory latency and minimizing stalls in the processor pipeline caused by memory access delays.
- However, its effectiveness depends on the predictability of memory access patterns and the efficiency of the prefetching mechanism employed by the processor.

Compiler-assisted Prefetching

- Some compilers can automatically analyze code and insert prefetch instructions based on the detected memory access patterns.
- This technique reduces the burden on the programmer, as the compiler takes care of prefetching optimizations.

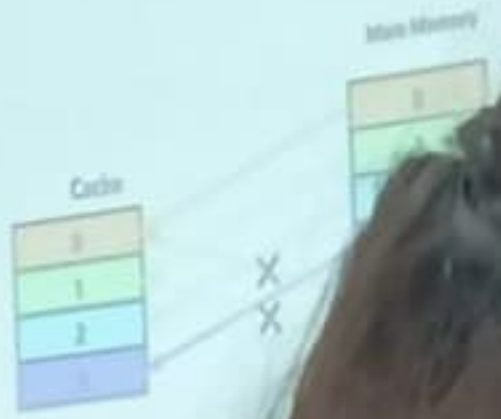
:43 AM Wed 19 Mar
All tools Edit Export E-Sign

Software Prefetching

- Programmers can manually insert prefetch instructions in their code to explicitly request the processor to fetch specific memory locations into the cache before they are accessed.
- This technique requires careful analysis of memory access patterns and an understanding of the underlying hardware.

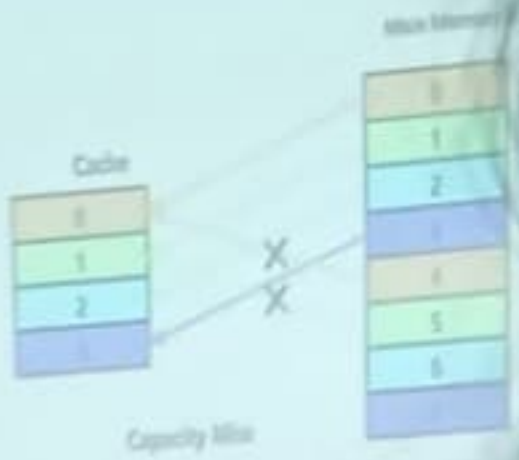
Capacity Cache Miss

- ***These misses occur when the program working set is much larger than the cache capacity.***
- Since Cache can not contain all blocks needed for program execution, so cache discards these blocks.
- Capacity misses occur low in direct mapped cache, medium in set associative cache, and high in associative mapped cache.
- **2:1 cache rule:** *a direct mapped cache of size N has the same miss rate as a 2-way set-associative cache of size N/2.*



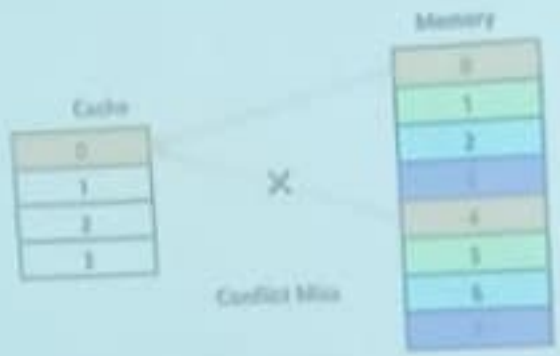
Capacity Cache Miss

- *These misses occur when the program working set is much larger than the cache capacity.*
- Since Cache can not contain all blocks needed for program execution, so cache discards these blocks.
- Capacity misses occur low in direct mapped cache, medium in set associative cache, and high in associative mapped cache.
- 2:1 cache rule: *a direct-mapped cache of size N has the same miss rate as a 2-way set-associative cache of size N/2.*



Conflict Cache Miss

- **It is also known as *collision miss* or *interference miss*.**
- These misses occur when several blocks are mapped to the same set or block frame.
- These misses occur in the set associative or direct mapped block placement strategies.
- *Conflict misses occur high in direct mapped cache, medium in set associative cache, and zero in associative mapped cache.*



Conflict Cache Miss

nown as ***collision miss or interference miss.***

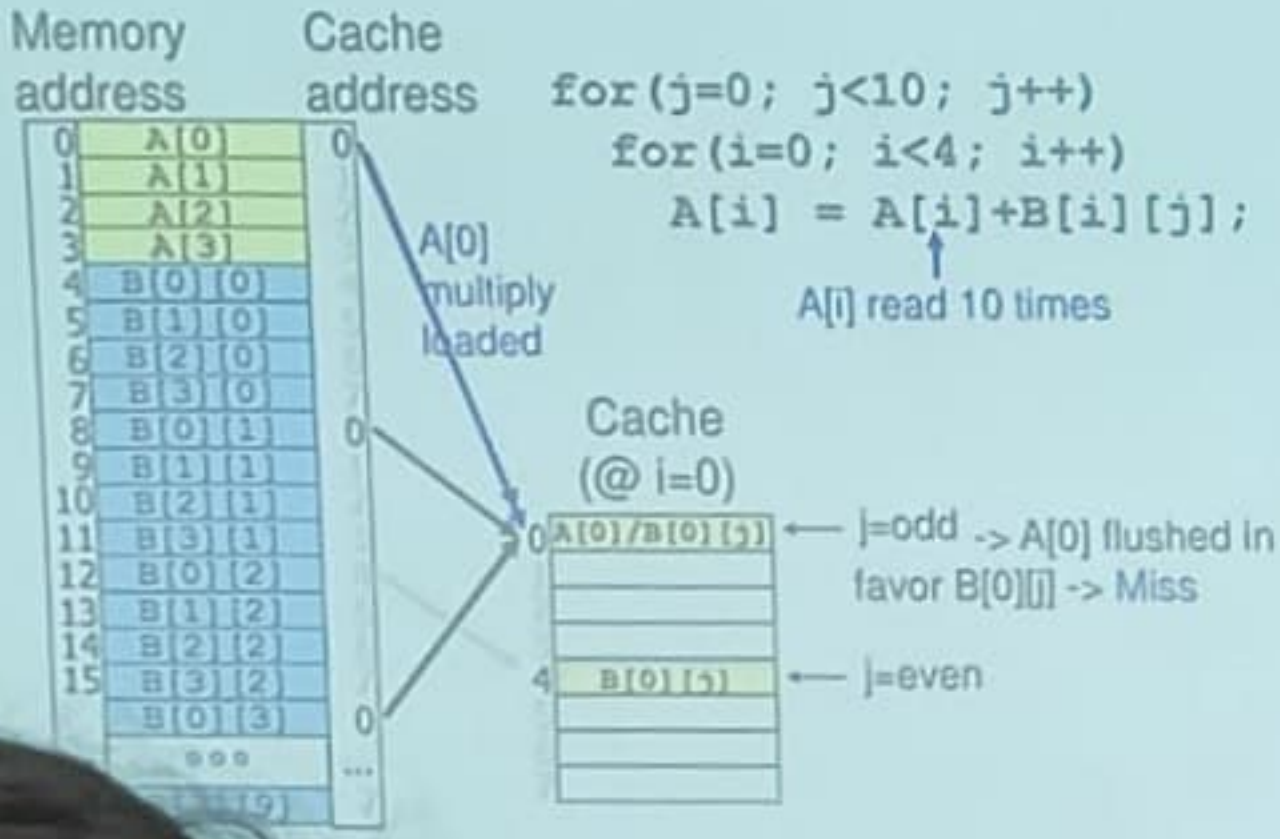
occur when several blocks are mapped to the same set or block frame.

occur in the set associative or direct mapped block placement strategies.

cur high in direct mapped cache, medium in set associative cache, and zero in

cache.

Conflict Cache Miss




```
for(j=0; j<10; j++)
  for(i=0; i<4; i++)
    A[i] = A[i]+B[i][j];
```

A[i] read 10 times

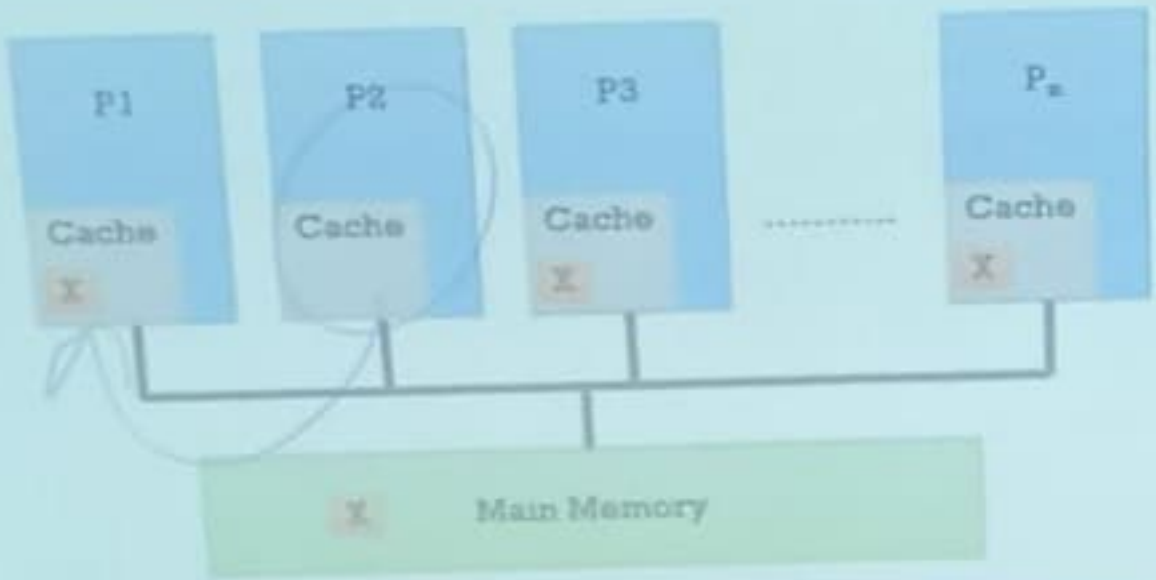
Coherence Cache Miss

- It is also known as ***Invalidation***.
- *These misses occur when other external processors update memory via I/O.*
- Coherence misses occur same in all types of direct mapped, set associative and associative caches.

(A nice and simple view of four processors and their shared address space)

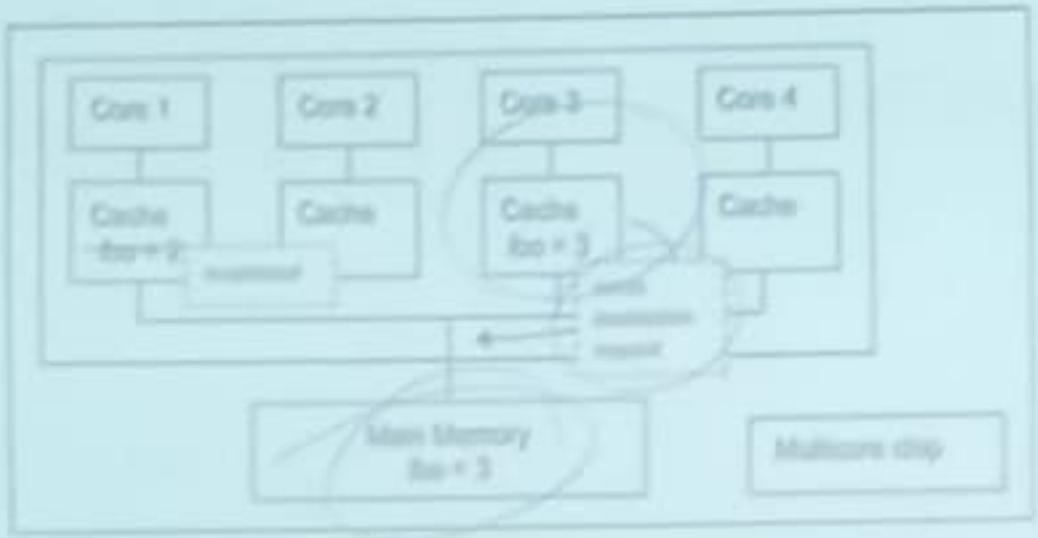
Edit Export E-Sign

Coherence Cache Miss



Scenario when processors shared data block and the data block is consistant

Coherence Cache Miss



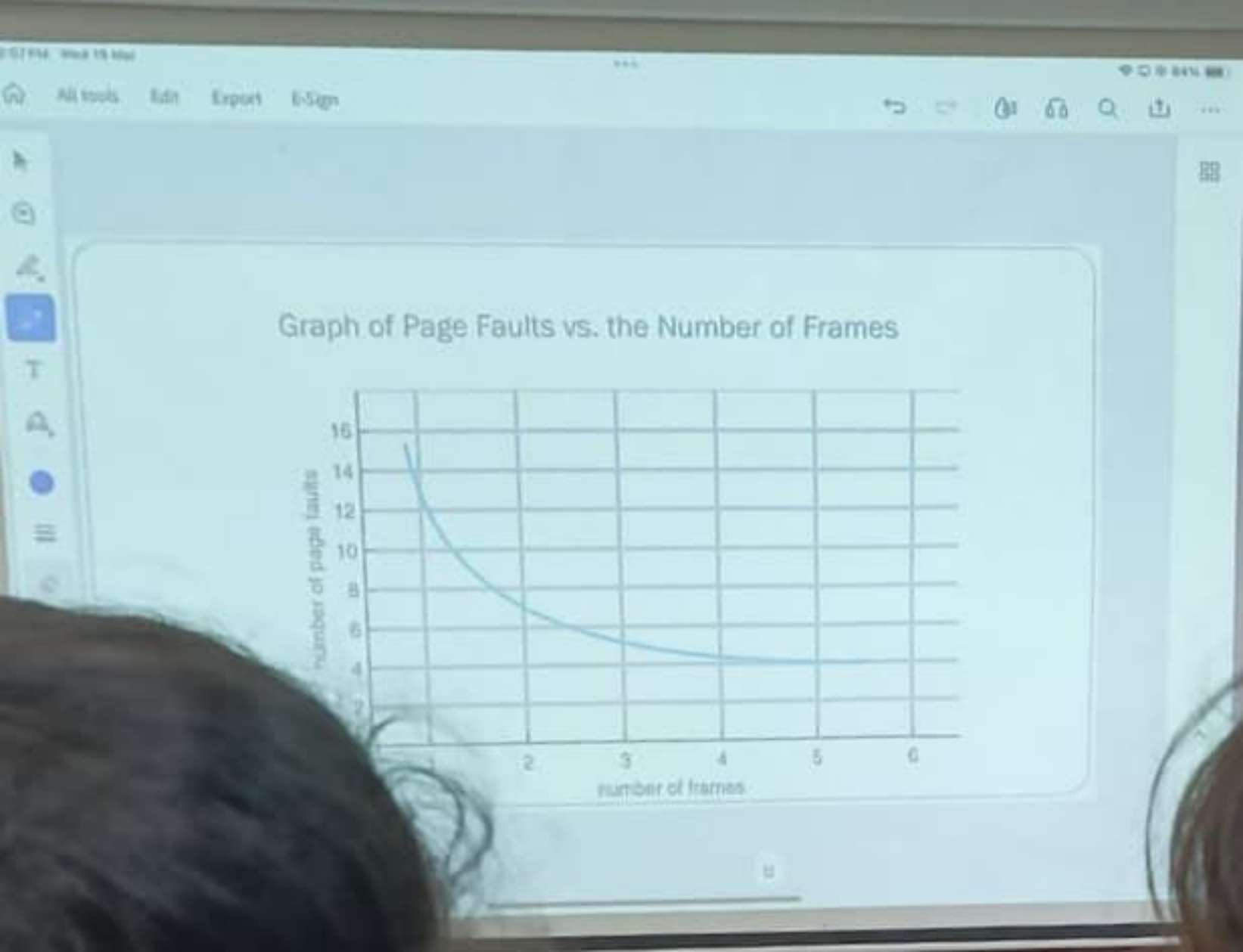
Solution? Replacement!!!

- One of the existing blocks is to be replaced when a new block is brought into cache.
- A cache replacement scheme or replacement policy is a method to replace an entry in the set by a new entry.
- Which block to be replaced on a cache miss?
  - Multiple replacement approaches

Find & Replace

Page Replacement Algorithms

- Targets for lowest page-fault rate.
- Evaluate algorithm by
  - running it on a particular string of memory references (reference string) and
  - computing the number of page faults and page replacements on that string
- In all our examples, we use a few recurring reference strings.


All tools
Edit
Export
E-Sign
Graph of Page Faults vs. the Number of Frames
number of page faults
16
14
12
10
8
6
4
2
2
3
4
5
6
number of frames

Replacement Schemes

- First-In-First-Out (FIFO) Algorithm
- LRU Algorithm
- LRU Approximation Algorithm
- Pseudo-LRU
- Random Replacement Scheme
- Optimal Algorithm
- Counting Algorithms
- LFU Algorithm
- MFU Algorithm

FIFO Page Replacement

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	2	4	4	4	0	0	0	7	7	7
	0	0	0	3	3	3	2	2	2	1	1	1	0	0
		1	1	1	0	0	0	3	3	3	2	2	2	1

First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process):

1	1	4	5	
2	2	1	3	9 page faults
3	3	2	4	

- 4 frames:

1	1	5	4	
2	2	1	5	10 page faults
3	3	2		
4	4	3		

- FIFO Replacement manifests Belady's Anomaly:
  more frames ⇒ more page faults

FIFO Illustrating Belady's Anomaly

Replacement Schemes

- First-In-First-Out (FIFO) Algorithm
- LRU Algorithm
- LRU Approximation Algorithm
- Pseudo-LRU
- Random Replacement Scheme
- Optimal Algorithm
- Counting Algorithms
- LFU Algorithm
- MFU Algorithm